

SINGAPORE-CAMBRIDGE H2 COMPUTING

NoSQL Paper 2 Practice Bank

LAB-BASED PRACTICAL QUESTIONS

SYLLABUS 9569

2026 ALIGNMENT

ORIGINAL PRACTICE

Paper 2 • Question Paper

Structured practice with progressive difficulty

Original practice material aligned mainly to syllabus outcome 3.3.8: use a programming language to work with NoSQL databases. It also practises validation, privacy and integrity from 3.3.9-3.3.10. Python 3, PyMongo and MongoDB match the lab software named in the 2026 syllabus.

This is a large practice bank, not one timed paper. It contains 12 independent sets and 60 tasks. Complete a set at a time. Suggested time is 45-70 minutes per set.

Task files are in the `task_files` folder beside this document. Use database `H2NoSQLPractice`. Use the collection named in each set. Start each set with a fresh collection unless instructed otherwise.

Difficulty: one star foundation; two stars standard; three stars challenge.

General requirements

- Use Python 3 and PyMongo.
- Convert CSV text to appropriate integer, float, Boolean, list or date values before insertion.
- Use unique identifiers in filters rather than names.
- Do not silently ignore invalid data.
- Close files with `with` and close the MongoDB client in a complete program.
- Display clear labels with results.
- Do not use `eval`.

Set 1 - Importing JSON and basic retrieval

Use `task_files/members.json` and collection `members`.

Task 1 (FOUNDATION) [4]

Write a complete program that connects to MongoDB, loads the JSON file with `json.load`, inserts all documents with one PyMongo method, displays the inserted count, and closes the client.

Task 2 (FOUNDATION) [4]

Display every member as `member_id - name - cca`. Do not display `_id`.

Task 3 (FOUNDATION) [4]

Ask for a CCA name and display the name and points of each active member in that CCA. Display `No matching members` when appropriate.

Task 4 (STANDARD) [4]

Display active members with at least 20 points, sorted by points descending and then name ascending. Retrieve only fields needed for the display.

Task 5 (STANDARD) [4]

Display the number of members in each CCA. Do the counting in Python, not with an aggregation pipeline.

Set 2 - Importing CSV with type conversion

Use `task_files/members_import.csv` and collection `members_csv`.

Task 6 (FOUNDATION) [4]

Write a program using `csv.DictReader` to insert the rows. Convert `year` and `points` to integers, `active` to Boolean, and split `skills` at `|` into an array.

Task 7 (STANDARD) [4]

Modify the importer to build a list and call `insert_many` once. Display the number inserted. Explain in a code comment one advantage over calling `insert_one` for every row.

Task 8 (STANDARD) [4]

Before importing, check whether any CSV `member_id` already exists in the collection. Skip duplicates and display each skipped identifier. Insert all other rows.

Task 9 (STANDARD) [4]

Display all inactive members and then display their count using `count_documents`.

Task 10 (CHALLENGE) [4]

Write `find_by_skill(collection, skill)` that returns a list of member names having the specified skill, sorted alphabetically. The function must return an empty list when none match.

Set 3 - Filters, projections and missing fields

Use `members.json` in collection `members`.

Task 11 (FOUNDATION) [4]

Write and run queries for: Year 2 members; members below 20 points; and members with Python in `skills`. Display identifiers and names.

Task 12 (STANDARD) [4]

Display active members in either Robotics or Media whose points are from 18 to 30 inclusive. Use one database query.

Task 13 (STANDARD) [4]

Display members that contain `contact.email`, showing name and email. Then display names of members without the field. Use dot notation and `$exists`.

Task 14 (STANDARD) [4]

Ask the user for minimum and maximum points. Validate that both are integers and minimum is not greater than maximum. Query and display matching documents or a clear error.

Task 15 (CHALLENGE) [4]

Write `search_members(collection, cca=None, active=None, skill=None)` that constructs a filter dynamically from only the non-None arguments and returns matching documents.

Set 4 - Safe updates and deletes

Use `members.json` in a fresh `members_updates` collection.

Task 16 (FOUNDATION) [4]

Set S003 to active and display the matched and modified counts. Retrieve S003 afterwards to prove the change.

Task 17 (STANDARD) [4]

Award 3 points to every active Robotics member with one update operation. Display how many records matched and then list the updated names and points.

Task 18 (STANDARD) [4]

Ask for a member ID and a new points value. Accept only integers 0-100. Use `$set`, report `Member not found`, and do not crash on non-numeric input.

Task 19 (STANDARD) [4]

Remove the `contact` field only from S001. Display the document before and after. Do not replace the whole document.

Task 20 (CHALLENGE) [4]

Write `safe_delete_inactive(collection, member_id)`. It may delete only if the exact member exists and is inactive. Return one of `deleted`, `active-member`, or `not-found`. Test all three paths.

Set 5 - Nested documents and arrays

Use `task_files/events.json` and collection events.

Task 21 (FOUNDATION) [4]

Import the file and display every non-cancelled event as `date title - block room` using fields inside `venue`.

Task 22 (STANDARD) [4]

Ask for a member ID and display all events for which the identifier occurs in `registered`. Show title and date.

Task 23 (STANDARD) [4]

Register S005 for E001 using an array update operator that does not create a duplicate. Display the array afterwards.

Task 24 (STANDARD) [4]

Remove S003 from E001's `registered` array using a single update operation. Display modified count and updated array.

Task 25 (CHALLENGE) [4]

Display events from least full to most full. For each, calculate registration percentage in Python as `len(registered) / capacity * 100`. Handle a capacity of zero safely even though the supplied file has none.

Set 6 - Attendance CSV and cross-collection work

Use `attendance.csv` in collection `attendance` and `members.json` in `members`.

Task 26 (FOUNDATION) [4]

Import `attendance`. Convert `present` to Boolean and `minutes` to integer. Display inserted count.

Task 27 (STANDARD) [4]

Ask for a member ID. Display sessions attended and total minutes. A record counts as attended only when `present` is true.

Task 28 (STANDARD) [4]

Display the attendance percentage for every member who has at least one attendance record. Calculate in Python and round to one decimal place.

Task 29 (CHALLENGE) [4]

Create a report showing member name rather than member ID for all absent records. Retrieve names from `members`. Avoid querying the `members` collection repeatedly inside the attendance loop.

Task 30 (CHALLENGE) [4]

Write a function that returns IDs of active members with no attendance record at all. Use both collections and Python set operations.

Set 7 - Inventory processing

Use `inventory.json` and collection `inventory`.

Task 31 (FOUNDATION) [4]

Import the data and display items whose quantity is less than or equal to reorder level, sorted by quantity ascending.

Task 32 (STANDARD) [4]

Calculate and display the total inventory value: the sum of `quantity * unit_cost` for every item. Display two decimal places.

Task 33 (STANDARD) [4]

Write `issue_item(collection, item_id, amount)`. Reject non-positive amounts, missing items and amounts greater than available stock. Otherwise atomically reduce quantity and return `issued`.

Task 34 (CHALLENGE) [4]

Improve Task 33 so the database filter itself prevents quantity going below zero if two users issue stock concurrently. Use the update result to decide whether the issue succeeded.

Task 35 (CHALLENGE) [4]

Write a restocking report containing item ID, name and `reorder_amount`, where the target quantity is twice the reorder level. Include only items at or below reorder level.

Set 8 - Loans and dates

Use `loans.csv` and collection `loans`.

Task 36 (FOUNDATION) [4]

Import `loans`, converting `returned` to Boolean. Store `loan_date` and `due_date` as Python `datetime` values rather than strings.

Task 37 (STANDARD) [4]

Given `today = datetime(2026, 7, 15)`, display all unreturned overdue loans with ID, member ID and days overdue.

Task 38 (STANDARD) [4]

Mark L004 as returned. Report not found, already returned, or successfully returned without producing a misleading success message.

Task 39 (CHALLENGE) [4]

Display the number of current unreturned loans per member, highest count first and member ID as the tie-breaker. Process the result in Python.

Task 40 (CHALLENGE) [4]

Write a function that attempts a new loan only when that member has fewer than three unreturned loans. Generate a new document from parameters and return a status string.

Set 9 - Embedded order items

Use `orders.json` and collection `orders`.

Task 41 (FOUNDATION) [4]

Import the orders and display paid orders for SCH02.

Task 42 (STANDARD) [4]

Display every order containing SKU P10. Use a query on a field inside the embedded array.

Task 43 (STANDARD) [4]

Calculate each order total in Python from `qty * price`. Display order ID and total to two decimal places.

Task 44 (CHALLENGE) [4]

Calculate total paid revenue per school, excluding pending and cancelled orders. Sort highest revenue first.

Task 45 (CHALLENGE) [4]

Write a function that adds an item to an order only if the order exists and is pending. Validate positive integer quantity and non-negative numeric price. Return a clear status.

Set 10 - Support-ticket reporting

Use `support_tickets.csv` and collection `tickets`.

Task 46 (FOUNDATION) [4]

Import tickets and display open or pending tickets with priority high or critical.

Task 47 (STANDARD) [4]

Display the number of tickets in each status. Perform counting in Python.

Task 48 (STANDARD) [4]

Ask for an assignee and display their unresolved tickets ordered with critical before high, medium and low. You may map priorities to numbers in Python.

Task 49 (CHALLENGE) [4]

Write `close_ticket(collection, ticket_id)` that refuses to close an already closed ticket, reports a missing ticket, and otherwise sets status to closed and records `closed_date` as the current UTC time.

Task 50 (CHALLENGE) [4]

Create a summary for each school containing total tickets, unresolved tickets and percentage resolved. Avoid division by zero.

Set 11 - Validation and dirty data

Use `dirty_members.csv`. Valid rows require a non-empty unique member ID, non-empty name, year 1 or 2, active exactly `true` or `false`, and integer points from 0 to 100.

Task 51 (STANDARD) [4]

Write `validate_row(row, seen_ids)` returning a list of error messages. It must check every rule rather than stop after the first error.

Task 52 (STANDARD) [4]

Write an importer that inserts valid rows into `validated_members` and writes invalid rows to the screen with line number and all reasons. Display valid and invalid counts.

Task 53 (CHALLENGE) [4]

Modify the importer so it also prevents duplicates already stored in MongoDB, not only duplicates within the CSV file.

Task 54 (STANDARD) [4]

Create a compound unique index on `school_id` and `member_id` for a multi-school collection. Catch and display a useful message if an insertion violates it.

Task 55 (CHALLENGE) [4]

Describe in code comments and implement two improvements that prevent a partially completed import from being mistaken for a successful complete import.

Set 12 - End-to-end challenge

Use any appropriate supplied task files in a new database `H2NoSQLChallenge`.

Task 56 (STANDARD) [4]

Write reusable `load_json(path)` and `load_csv(path)` functions. The CSV loader returns raw dictionaries; conversion remains scenario-specific. Handle missing files and invalid JSON with helpful messages.

Task 57 (CHALLENGE) [4]

Write a menu-driven member application with options to search, add, edit points, deactivate and quit. Validate all inputs and use member IDs for updates. Separate database operations into functions.

Task 58 (CHALLENGE) [4]

Write a reporting program that combines `members`, `attendance` and `loans`. For each active member display name, attendance percentage and number of unreturned loans. Avoid one database query per member per collection.

Task 59 (CHALLENGE) [4]

Add audit logging to member point changes. Every successful update must insert a document into `audit_log` containing member ID, old value, new value, timestamp and action. Do not log failed updates.

Task 60 (CHALLENGE) [4]

Design and implement a test driver for Tasks 57-59. Seed a test database, exercise normal/boundary/invalid/not-found cases, check stored results with assertions, print a pass/fail summary, and remove only the named test collections when finished.

End of question bank. Total available: 240 marks.